

Documentacao completa do projeto EvolutionSender

Projeto EvolutionSender - documentacao consolidada de membros, mensagens e envio via Evolution API.

Este documento consolida a historia tecnica do projeto `EvolutionSender`, desde a base inicial de CRUD de membros ate a migracao do fluxo de envio de mensagens do n8n para dentro do ASP.NET Core MVC, usando Evolution API.

O objetivo e manter um unico registro detalhado do que foi construido, por que foi construido e como cada parte se conecta.

1. Visao geral

O `EvolutionSender` e uma aplicacao ASP.NET Core MVC criada para apoiar a comunicacao da RedeUp com jovens/membros cadastrados em banco MySQL/MariaDB.

A aplicacao hoje possui:

- ? CRUD de membros;
- ? CRUD de mensagens;
- ? envio manual de mensagens via Evolution API;
- ? envio individual de uma mensagem especifica;
- ? envio em lote de todas as mensagens ativas;
- ? delay configuravel entre envios;
- ? logs de erro;
- ? protecao visual contra clique duplo na tela;
- ? inativacao automatica de mensagens enviadas.

2. Tecnologias usadas

Stack principal:

- ? ASP.NET Core MVC;
- ? .NET 9;
- ? MySQL/MariaDB;
- ? Dapper;
- ? MySqlConnection;
- ? Bootstrap;
- ? JavaScript simples em `wwwroot/js/site.js`;
- ? Evolution API para envio de WhatsApp.

Pacotes principais no projeto:

```
<PackageReference Include="Dapper" Version="2.1.72" />
<PackageReference Include="MySqlConnection" Version="2.5.0" />
<PackageReference Include="Newtonsoft.Json" Version="13.0.4" />
```

3. Estrutura geral do projeto

Pastas principais:

```
Controllers/  
Data/  
Models/  
Services/  
Views/  
wwwroot/  
docs/
```

Responsabilidade de cada pasta:

- ? **Controllers**: recebe requisicoes HTTP e chama repositorios/servicos;
- ? **Data**: concentra acesso ao banco com Dapper;
- ? **Models**: classes que representam dados usados pelas telas e repositorios;
- ? **Services**: regras de envio, cliente Evolution API e opcoes de configuracao;
- ? **Views**: telas Razor MVC;
- ? **wwwroot**: arquivos estaticos, como CSS e JavaScript;
- ? **docs**: documentacao do projeto.

4. Configuracao do banco

A connection string principal foi configurada em `appsettings.json` e `appsettings.Development.json`.

```
"ConnectionStrings": {  
  "RedeUp": "Server=127.0.0.1;Port=3307;Database=redeup;User ID=root;Password=root@cesso;"  
}
```

Essa string e usada pelos repositorios:

- ? `MembroRepository`;
- ? `MensagemRepository`.

5. Tabela de membros

Tabela usada:

```
menbros
```

Observacao importante: o nome da tabela no banco esta como `menbros`, com n.

Campos usados:

```
CREATE TABLE menbros (  
  codigo INT NOT NULL AUTO_INCREMENT,  
  nome VARCHAR(150) NOT NULL,  
  data_de_nascimento DATE NOT NULL,  
  telefone VARCHAR(20) NULL,  
  observacao VARCHAR(255) NULL,  
  primeiro_nome VARCHAR(100) GENERATED ALWAYS AS (...),  
  ativo INT NOT NULL DEFAULT 1,  
  PRIMARY KEY (codigo)  
);
```

6. CRUD de membros

O primeiro grande bloco do projeto foi o cadastro de membros da RedeUp.

Funcionalidades implementadas:

- ? listar membros ativos;
- ? buscar por nome, primeiro nome ou telefone;
- ? incluir inativos na busca;
- ? cadastrar novo membro;
- ? editar membro;
- ? visualizar detalhes;
- ? inativar membro.

O sistema não apaga fisicamente membros. A exclusão lógica altera o campo:

```
ativo = 0
```

7. Model de membros

Arquivo:

```
Models/Membro.cs
```

Propriedades principais:

```
public int Codigo { get; set; }
public string Nome { get; set; } = string.Empty;
public DateTime DataDeNascimento { get; set; }
public string? Telefone { get; set; }
public string? Observacao { get; set; }
public string PrimeiroNome { get; set; } = string.Empty;
public int Ativo { get; set; } = 1;
public bool EstaAtivo => Ativo == 1;
```

Observação: `PrimeiroNome` é lido do banco como coluna gerada. Por isso, o sistema não envia valor para esse campo no cadastro nem na edição.

8. Repositorio de membros

Arquivos:

```
Data/IMembroRepository.cs
Data/MembroRepository.cs
```

Métodos criados:

```
Task<IReadOnlyList<Membro>> ListarAsync(string? busca, bool incluirInativos);
Task<Membro?> ObterPorCodigoAsync(int codigo);
Task<int> CriarAsync(Membro membro);
Task<bool> AtualizarAsync(Membro membro);
Task<bool> ExcluirAsync(int codigo);
```

O método `ListarAsync` busca membros ativos por padrão:

```
FROM membros
WHERE (@IncluirInativos = TRUE OR ativo = 1)
```

Durante testes de envio foi usado um filtro temporário no repositório para restringir o envio a um único cadastro:

```
WHERE codigo = 37
```

Esse filtro foi usado para garantir que as mensagens fossem enviadas apenas para o numero de teste.

9. Controller e telas de membros

Controller:

```
Controllers/MembrosController.cs
```

Rotas principais:

```
/Membros  
/Membros/Details/{id}  
/Membros/Create  
/Membros/Edit/{id}  
/Membros/Delete/{id}
```

Views:

```
Views/Membros/Index.cshtml  
Views/Membros/Create.cshtml  
Views/Membros/Edit.cshtml  
Views/Membros/Details.cshtml  
Views/Membros/Delete.cshtml  
Views/Membros/_Form.cshtml
```

10. Tabela de mensagens

Depois do CRUD de membros, foi criado o CRUD de mensagens com base na tabela:

```
mensagens
```

Campos:

```
id  
titulo  
ocasio  
mensagem  
ativo  
criado_em  
atualizado_em
```

Objetivo da tabela:

- ? cadastrar textos de comunicacao;
- ? revisar mensagens antes de enviar;
- ? deixar apenas mensagens prontas com `ativo = 1`;
- ? inativar mensagens depois do envio para evitar duplicidade.

11. Model de mensagens

Arquivo:

```
Models/Mensagem.cs
```

Propriedades principais:

```
public int Id { get; set; }  
public string Titulo { get; set; } = string.Empty;
```

```

public string Ocasiao { get; set; } = string.Empty;
public string Texto { get; set; } = string.Empty;
public int Ativo { get; set; } = 1;
public DateTime? CriadoEm { get; set; }
public DateTime? AtualizadoEm { get; set; }
public bool EstaAtiva => Ativo == 1;

```

Observacao: no banco o campo se chama `mensagem`, mas no C# ele foi mapeado como `Texto`.

12. CRUD de mensagens

Arquivos principais:

```

Data/IMensagemRepository.cs
Data/MensagemRepository.cs
Controllers/MensagensController.cs
Views/Mensagens/

```

Funcionalidades do CRUD:

- ? listar mensagens;
- ? filtrar por titulo, ocasiao ou texto;
- ? incluir inativas na listagem;
- ? cadastrar mensagem;
- ? editar mensagem;
- ? visualizar detalhes;
- ? inativar mensagem.

Assim como no CRUD de membros, a exclusao de mensagens e logica. O sistema nao remove a linha da tabela, apenas altera:

```

ativo = 0

```

13. Repositorio de mensagens

Arquivos:

```

Data/IMensagemRepository.cs
Data/MensagemRepository.cs

```

Metodos principais:

```

Task<IReadOnlyList<Mensagem>> ListarAsync(string? busca, bool incluirInativas);
Task<IReadOnlyList<Mensagem>> ListarAtivasAsync();
Task<Mensagem?> ObterAtivaAsync();
Task<Mensagem?> ObterAtivaPorIdAsync(int id);
Task<Mensagem?> ObterPorIdAsync(int id);
Task<int> CriarAsync(Mensagem mensagem);
Task<bool> AtualizarAsync(Mensagem mensagem);
Task<bool> InativarAsync(int id);
Task<bool> ExcluirAsync(int id);

```

13.1. Listar mensagens

Usado na tela principal de mensagens:

```

SELECT
    id AS Id,
    titulo AS Titulo,
    ocasiao AS Ocasiao,

```

```

        mensagem AS Texto,
        ativo AS Ativo,
        criado_em AS CriadoEm,
        atualizado_em AS AtualizadoEm
FROM mensagens
WHERE (@IncluirInativas = TRUE OR ativo = 1)
AND (
    @Busca IS NULL
    OR titulo LIKE CONCAT('%', @Busca, '%')
    OR ocasiao LIKE CONCAT('%', @Busca, '%')
    OR mensagem LIKE CONCAT('%', @Busca, '%')
)
ORDER BY criado_em DESC, id DESC;

```

13.2. Listar todas as mensagens ativas

Usado pelo botao da listagem **Enviar mensagens ativas**.

```

SELECT
    id AS Id,
    titulo AS Titulo,
    ocasiao AS Ocasiao,
    mensagem AS Texto,
    ativo AS Ativo,
    criado_em AS CriadoEm,
    atualizado_em AS AtualizadoEm
FROM mensagens
WHERE ativo = 1
ORDER BY atualizado_em ASC, criado_em ASC, id ASC;

```

Esse metodo permite processar todas as mensagens pendentes em loop.

13.3. Buscar mensagem ativa por id

Usado pelo botao interno da tela de detalhes.

```

SELECT
    id AS Id,
    titulo AS Titulo,
    ocasiao AS Ocasiao,
    mensagem AS Texto,
    ativo AS Ativo,
    criado_em AS CriadoEm,
    atualizado_em AS AtualizadoEm
FROM mensagens
WHERE ativo = 1
AND id = @Id
ORDER BY atualizado_em DESC
LIMIT 1;

```

Esse comportamento reproduz o filtro que existia no n8n:

```

WHERE ativo = 1
AND id = [{ $json.id }]

```

13.4. Inativar mensagem

Usado depois do envio:

```

UPDATE mensagens
SET ativo = 0,
    atualizado_em = CURRENT_TIMESTAMP
WHERE id = @Id;

```

14. Fluxo original no n8n

Antes da migracao, o n8n executava uma consulta parecida com:

```

SELECT
    c.codigo,
    c.nome,
    SUBSTRING_INDEX(TRIM(c.nome), ' ', 1) AS primeiro_nome,
    CASE
        WHEN REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(TRIM(c.telefone), ' ', ''), '- ', ''), '(', ''), ')', '' ), '+', '' ) LIKE '55%'
        THEN REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(TRIM(c.telefone), ' ', ''), '- ', ''), '(', ''), ')', '' ), '+', '' )
        ELSE CONCAT('5522', REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(REPLACE(TRIM(c.telefone), ' ', ''), '- ', ''), '(', ''), ')', '' ), '+', '' ))
        END AS telefone,
    m.mensagem_id,
    m.titulo,
    m.texto_base
FROM menbros c
CROSS JOIN (
    SELECT
        id AS mensagem_id,
        titulo,
        mensagem AS texto_base
    FROM mensagens
    WHERE ativo = 1
    AND id = {{ $json.id }}
    ORDER BY atualizado_em DESC
    LIMIT 1
) m
WHERE c.telefone IS NOT NULL
    AND TRIM(c.telefone) != ''
    AND c.ativo = 1;

```

O ASP.NET passou a assumir essas responsabilidades.

15. Migracao do n8n para ASP.NET

Objetivo da migracao:

- ? tirar a logica principal do n8n;
- ? deixar o envio dentro do proprio sistema;
- ? manter a tela como ponto manual de controle;
- ? organizar a regra em servicos;
- ? preservar a logica de mensagem ativa;
- ? evitar envio duplicado inativando mensagens ao final.

Foram criados servicos para separar responsabilidades:

```

Services/IEvolutionApiClient.cs
Services/EvolutionApiClient.cs
Services/EvolutionApiOptions.cs
Services/EvolutionApiSendResult.cs
Services/IEnvioMensagemService.cs
Services/EnvioMensagemService.cs
Services/EnvioMensagemResultado.cs
Services/EnvioMensagemOptions.cs

```

16. Configuracao da Evolution API

Secao adicionada aos arquivos de configuracao:

```

"EvolutionApi": {
  "BaseUrl": "http://localhost:8080",
  "InstanceName": "igreja",
  "ApiKey": "CONFIGURADO_LOCALMENTE",
  "ApiKeyHeaderName": "apikey",
  "SendTextEndpointTemplate": "/message/sendText/{instance}"
}

```

```
}
```

Base URL local validada:

```
http://localhost:8080
```

Instancia:

```
igreja
```

Endpoint chamado:

```
POST http://localhost:8080/message/sendText/igreja
```

Header:

```
apikey: valor-configurado
```

Corpo enviado:

```
{
  "number": "5522...",
  "text": "mensagem"
}
```

Recomendacao: a API key usada localmente nao deve ser versionada em repositorio publico. Em uma etapa futura, mover para user secrets, variavel de ambiente ou cofre de segredos.

17. Cliente da Evolution API

Interface:

```
Services/IEvolutionApiClient.cs

public interface IEvolutionApiClient
{
    Task<EvolutionApiSendResult> EnviarTextoAsync(
        string numero,
        string texto,
        CancellationToken cancellationToken = default);
}
```

Implementacao:

```
Services/EvolutionApiClient.cs
```

Responsabilidades:

- ? validar se `BaseUrl`, `InstanceName` e `ApiKey` foram configurados;
- ? montar o endpoint substituindo `{instance}`;
- ? criar a requisicao HTTP POST;
- ? adicionar header `apikey`;
- ? enviar JSON com `number` e `text`;
- ? retornar sucesso ou erro padronizado.

18. Servico de envio de mensagens

Interface:

Services/IEnvioMensagemService.cs

Metodos expostos:

```
Task<EnvioMensagemResultado> EnviarMensagemAtivaAsync(
    CancellationToken cancellationToken = default);

Task<EnvioMensagemResultado> EnviarTodasMensagensAtivasAsync(
    CancellationToken cancellationToken = default);

Task<EnvioMensagemResultado> EnviarMensagemAtivaAsync(
    int mensagemId,
    CancellationToken cancellationToken = default);
```

Implementacao:

Services/EnvioMensagemService.cs

Responsabilidades:

- ? buscar mensagem ativa;
- ? buscar todas as mensagens ativas;
- ? buscar membros ativos;
- ? normalizar telefone;
- ? montar texto personalizado;
- ? chamar Evolution API;
- ? aguardar delay entre envios;
- ? registrar logs;
- ? inativar mensagem enviada;
- ? retornar resumo para a tela.

19. Envio individual e envio em lote

Existem dois comportamentos diferentes.

19.1. Botao da listagem

Texto:

Enviar mensagens ativas

Local:

Views/Mensagens/Index.cshtml

Comportamento:

- ? nao envia id;
- ? o controller chama EnviarTodasMensagensAtivasAsync;
- ? o sistema lista todas as mensagens com ativo = 1;
- ? envia cada mensagem para os membros ativos;
- ? inativa cada mensagem que teve pelo menos um envio com sucesso;
- ? aplica delay entre membros;

? aplica delay entre uma mensagem ativa e a proxima.

19.2. Botao interno de detalhes

Texto:

Enviar

Local:

Views/Mensagens/Details.cshtml

Comportamento:

- ? envia o `id` da mensagem em campo hidden;
- ? o controller chama `EnviarMensagemAtivaAsync(id)`;
- ? o sistema envia apenas aquela mensagem;
- ? se a mensagem estiver ativa, o botao fica verde;
- ? se a mensagem estiver inativa, o botao fica cinza e desabilitado.

Campo usado:

```
<input type="hidden" name="id" value="@Model.Id" />
```

Regra no controller:

```
var resultado = id.HasValue
    ? await _envioMensagemService.EnviarMensagemAtivaAsync(id.Value, cancellationToken)
    : await _envioMensagemService.EnviarTodasMensagensAtivasAsync(cancellationToken);
```

20. Regra de telefone

A regra de telefone foi migrada da query do n8n para C#.

Regra atual:

1. Remove tudo que nao for digito.
2. Se o numero comeca com 55, mantem.
3. Se nao comeca com 55, adiciona 5522.

Exemplos:

```
(22) 99999-9999 -> 5522999999999
+55 22 99999-9999 -> 5522999999999
2299999999999 -> 5522999999999
```

Metodo:

```
private static string? NormalizarTelefone(string? telefone)
```

21. Montagem do texto

O texto base vem da coluna:

`mensagens.mensagem`

No model:

```
Mensagem.Texto
```

Placeholders suportados:

```
{nome}  
{primeiro_nome}  
{telefone}
```

Exemplo de mensagem:

```
Ola, {primeiro_nome}! Hoje temos encontro.
```

Resultado para membro Rafael:

```
Ola, Rafael! Hoje temos encontro.
```

Método:

```
private static string MontarTexto(Mensagem mensagem, Membro membro)
```

22. Delay entre envios

Foi adicionada uma configuração para reduzir o risco de envio em massa muito rápido.

Arquivo:

```
Services/EnvioMensagemOptions.cs
```

Classe:

```
public class EnvioMensagemOptions  
{  
    public int DelayMinimoSegundos { get; set; } = 5;  
    public int DelayMaximoSegundos { get; set; } = 10;  
}
```

Configuração:

```
"EnvioMensagem": {  
    "DelayMinimoSegundos": 5,  
    "DelayMaximoSegundos": 10  
}
```

Comportamento:

- ? após cada tentativa real de envio para um membro, aguarda entre 5 e 10 segundos;
- ? após terminar uma mensagem ativa e antes de iniciar a próxima, também aguarda entre 5 e 10 segundos;
- ? o delay não é aplicado após o último membro da lista;
- ? o delay não garante ausência de bloqueio pelo WhatsApp, mas reduz disparos muito rápidos.

23. Logs de erro

O serviço usa:

```
ILogger<EnvioMensagemService>
```

23.1. Telefone inválido

Quando o telefone esta vazio ou invalido:

```
Envio ignorado por telefone invalido.  
Codigo: {Codigo}  
Nome: {Nome}  
TelefoneOriginal: {TelefoneOriginal}
```

23.2. Falha na Evolution API

Quando a Evolution API retorna erro:

```
Falha ao enviar mensagem.  
MensagemId: {MensagemId}  
Codigo: {Codigo}  
Nome: {Nome}  
Numero: {Numero}  
Erro: {Erro}
```

O erro tambem e retornado para a tela. A tela mostra ate 10 detalhes usando:

```
TempData["DetalhesErro"]
```

24. Resultado do envio

Arquivo:

```
Services/EnvioMensagemResultado.cs
```

Classe atual:

```
public class EnvioMensagemResultado  
{  
    public int MensagemId { get; init; }  
    public int TotalMensagens { get; init; }  
    public int TotalMensagensInativadas { get; init; }  
    public int TotalMembros { get; init; }  
    public int TotalEnviados { get; init; }  
    public int TotalErros { get; init; }  
    public bool MensagemInativada { get; init; }  
    public IReadOnlyList<string> Erros { get; init; } = [];  
  
    public bool Sucesso => TotalEnviados > 0  
        && TotalErros == 0  
        && MensagemInativada;  
}
```

Resumo mostrado na tela quando houve envio:

```
Envio finalizado: X/Y mensagem(ns) processada(s), Z envio(s), W erro(s).
```

25. Controller de mensagens

Arquivo:

```
Controllers/MensagensController.cs
```

Foi injetado:

```
private readonly IEnvioMensagemService _envioMensagemService;
```

Acao de envio:

```
[HttpPost]  
[ValidateAntiForgeryToken]
```

```
public async Task<IActionResult> EnviarAtiva(
    int? id,
    CancellationToken cancellationToken)
```

Comportamento:

- ? se `id` tem valor, envia apenas aquela mensagem;
- ? se `id` não tem valor, envia todas as mensagens ativas;
- ? se houve envio, mostra resumo de sucesso;
- ? se não houve envio, mostra erro;
- ? se houver erros, mostra os primeiros 10 detalhes;
- ? se o envio veio da tela de detalhes, redireciona de volta para detalhes;
- ? se veio da listagem, redireciona para a listagem.

26. Interface de usuario

26.1. Listagem de mensagens

Arquivo:

```
Views/Mensagens/Index.cshtml
```

Elementos principais:

- ? título `Mensagens`;
- ? botão `Enviar mensagens ativas`;
- ? botão `Nova mensagem`;
- ? alerta de sucesso;
- ? alerta de erro;
- ? alerta com detalhes de erro;
- ? filtro por texto;
- ? checkbox `Incluir inativas`;
- ? tabela de mensagens.

O botão `Enviar mensagens ativas`:

- ? envia todas as mensagens ativas;
- ? usa spinner de carregamento;
- ? fica desabilitado após o clique;
- ? evita clique duplo enquanto a requisição está em andamento.

26.2. Detalhes da mensagem

Arquivo:

```
Views/Mensagens/Details.cshtml
```

Elementos principais:

- ? botão `Voltar`;

- ? botao `Enviar`;
- ? botao `Editar`;
- ? dados da mensagem;
- ? texto completo da mensagem.

Regra visual do botao `Enviar`:

- ? mensagem ativa: botao verde e clicavel;
- ? mensagem inativa: botao cinza e desabilitado.

27. Trava contra clique duplo e spinner

Arquivo:

```
wwwroot/js/site.js
```

Foi adicionado JavaScript para formularios marcados com:

```
data-form-envio-mensagem
```

Comportamento ao submeter:

- ? localiza o botao de envio;
- ? desabilita o botao;
- ? adiciona `aria-busy="true"`;
- ? exhibe spinner Bootstrap;
- ? troca o texto para `Enviando...`

Trecho conceitual:

```
botao.disabled = true;
botao.setAttribute('aria-busy', 'true');
spinner.classList.remove('d-none');
texto.textContent = 'Enviando...';
```

28. Injecao de dependencia

Arquivo:

```
Program.cs
```

Registros principais:

```
builder.Services.AddScoped<IMembroRepository, MembroRepository>();
builder.Services.AddScoped<IMensagemRepository, MensagemRepository>();

builder.Services.Configure<EvolutionApiOptions>(
    builder.Configuration.GetSection("EvolutionApi"));

builder.Services.Configure<EnvioMensagemOptions>(
    builder.Configuration.GetSection("EnvioMensagem"));

builder.Services.AddScoped<IEnvioMensagemService, EnvioMensagemService>();

builder.Services.AddHttpClient<IEvolutionApiClient, EvolutionApiClient>(
    (serviceProvider, client) =>
    {
        var options = serviceProvider
            .GetRequiredService<IOptions<EvolutionApiOptions>>()
```

```

        .Value;

        if (!string.IsNullOrEmpty(options.BaseUrl))
        {
            client.BaseAddress = new Uri(options.BaseUrl);
        }
    });

```

O logging tambem foi ajustado:

```

builder.Logging.ClearProviders();
builder.Logging.AddConsole();
builder.Logging.AddDebug();

```

Isso evita dependencia do Log de Eventos do Windows durante o desenvolvimento.

29. Fluxo completo atual

29.1. Envio de todas as mensagens ativas

```

Usuario clica em Enviar mensagens ativas
|
v
MensagensController.EnviarAtiva sem id
|
v
EnvioMensagemService.EnviaTodasMensagensAtivasAsync
|
v
MensagemRepository.ListarAtivasAsync
|
v
Para cada mensagem ativa:
|
+--> buscar membros ativos
+--> normalizar telefone
+--> montar texto
+--> chamar Evolution API
+--> registrar sucesso/erro
+--> aguardar delay entre membros
+--> inativar mensagem se houve envio
+--> aguardar delay antes da proxima mensagem
|
v
Controller mostra resumo final

```

29.2. Envio de uma mensagem especifica

```

Usuario entra nos detalhes da mensagem
|
v
Clica em Enviar
|
v
Formulario envia id da mensagem
|
v
MensagensController.EnviarAtiva com id
|
v
EnvioMensagemService.EnviaMensagemAtivaAsync(id)
|
v
MensagemRepository.ObterAtivaPorIdAsync(id)
|
v
Envia apenas aquela mensagem
|
v
Inativa se houve envio com sucesso

```

30. Validacoes realizadas

Comando usado para validar compilacao:

```
dotnet build --no-restore
```

Resultado obtido apos as alteracoes:

```
Compilacao com sucesso.  
0 Aviso(s)  
0 Erro(s)
```

Tambem foi feita validacao de inicializacao em porta temporaria:

```
dotnet run --no-build --no-launch-profile --urls http://localhost:5241
```

O app iniciou e escutou na porta informada.

Observacao: em algumas execucoes no ambiente do assistente apareceu erro de permissao ao acessar chaves internas do ASP.NET em AppData, mas a aplicacao iniciou normalmente. No uso local normal, isso nao bloqueou o fluxo.

31. Testes funcionais realizados

Testes informados como bem-sucedidos:

- ? envio via Evolution API funcionando;
- ? envio usando instancia igreja;
- ? Evolution API respondendo em `http://localhost:8080`;
- ? envio limitado ao cadastro de teste `codigo = 37`;
- ? delay/logs implementados;
- ? spinner e trava contra clique duplo funcionando;
- ? botao interno `Enviar` funcionando;
- ? botao interno respeitando mensagem ativa/inativa.

32. Como executar o projeto

Compilar:

```
dotnet build --no-restore
```

Rodar:

```
dotnet run --urls http://localhost:5238
```

Abrir telas principais:

```
http://localhost:5238/Membros  
http://localhost:5238/Mensagens
```

Tambem existe o atalho:

```
Abrir EvolutionSender.cmd
```

33. Cuidados operacionais

33.1. Evitar envio acidental para todos

Durante testes, manter filtro temporario no `MembroRepository`:

```
WHERE codigo = 37
```

Antes de enviar para todos:

- ? remover o filtro de teste;
- ? revisar membros ativos;
- ? revisar mensagens ativas;
- ? confirmar a instancia conectada na Evolution API;
- ? confirmar o delay desejado.

33.2. Fechar instancia antes de compilar

Quando o app esta rodando, o Windows pode bloquear:

```
bin/Debug/net9.0/EvolutionSender.exe
```

Se isso acontecer, fechar a instancia do `EvolutionSender` antes de compilar.

33.3. Cuidado com a API key

A chave da Evolution API foi configurada localmente para teste.

Antes de versionar ou compartilhar:

- ? remover a chave do arquivo;
- ? usar user secrets;
- ? ou usar variavel de ambiente.

33.4. Delay nao elimina risco de bloqueio

O delay entre 5 e 10 segundos reduz risco de envio rapido demais, mas nao garante que o WhatsApp nunca limite ou bloqueie uma instancia.

Boas praticas:

- ? usar intervalos maiores em listas grandes;
- ? evitar muitas mensagens identicas;
- ? acompanhar logs de erro;
- ? validar numeros antes do envio;
- ? criar historico persistente de envios;
- ? considerar fila/background job futuramente.

34. Arquivos principais alterados/criados

Models:

```
Models/Membro.cs
```

Models/Mensagem.cs

Data:

Data/IMembroRepository.cs
Data/MembroRepository.cs
Data/IMensagemRepository.cs
Data/MensagemRepository.cs

Controllers:

Controllers/MembrosController.cs
Controllers/MensagensController.cs

Services:

Services/IEvolutionApiClient.cs
Services/EvolutionApiClient.cs
Services/EvolutionApiOptions.cs
Services/EvolutionApiSendResult.cs
Services/IEnvioMensagemService.cs
Services/EnvioMensagemService.cs
Services/EnvioMensagemResultado.cs
Services/EnvioMensagemOptions.cs

Views:

Views/Membros/
Views/Mensagens/
Views/Shared/_Layout.cshtml
Views/Home/Index.cshtml

Configuracao e frontend:

Program.cs
appsettings.json
appsettings.Development.json
wwwroot/js/site.js
.gitignore
Abrir EvolutionSender.cmd

Documentacao:

docs/script-crud-membros.md
docs/fluxo-envio-evolution-api.md
docs/fluxo-envio-evolution-api.pdf
docs/documentacao-completa-evolutionsender.md
docs/documentacao-completa-evolutionsender.pdf

35. Proximos passos recomendados

35.1. Criar tabela de historico de envios

Sugestao:

```
CREATE TABLE envios_mensagens (  
    id INT UNSIGNED NOT NULL AUTO_INCREMENT,  
    mensagem_id INT UNSIGNED NOT NULL,  
    membro_codigo INT NOT NULL,  
    telefone VARCHAR(30) NOT NULL,  
    status VARCHAR(30) NOT NULL,  
    erro TEXT NULL,  
    criado_em TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (id)  
);
```

Beneficios:

- ? saber quem recebeu;
- ? saber quem falhou;
- ? registrar resposta da Evolution API;
- ? permitir reenvio apenas das falhas;
- ? auditar envios antigos.

35.2. Criar tela de resultado detalhado

Hoje o resultado aparece em alerta simples.

Uma tela dedicada poderia mostrar:

- ? mensagens processadas;
- ? membros encontrados;
- ? envios com sucesso;
- ? envios com erro;
- ? numeros invalidos;
- ? botao para reenviar falhas.

35.3. Migrar segredo para user secrets

Usar:

```
dotnet user-secrets
```

ou variaveis de ambiente.

35.4. Processar em background futuramente

Hoje o envio ocorre durante a requisicao HTTP.

Para listas grandes, o ideal sera:

- ? fila de envio;
- ? worker em background;
- ? tela de progresso;
- ? pausa/cancelamento;
- ? historico persistente;
- ? protecao contra timeout do navegador.

36. Estado atual

No estado atual, o projeto permite:

- ? gerenciar membros;
- ? gerenciar mensagens;
- ? enviar uma mensagem especifica pela tela de detalhes;
- ? enviar todas as mensagens ativas pela listagem;

- ? inativar mensagens apos envio;
- ? aguardar delay entre disparos;
- ? registrar erros no log;
- ? evitar clique duplo no botao de envio;
- ? diferenciar visualmente mensagem ativa e inativa.

Este documento substitui, como referencia principal, os documentos separados anteriores.